Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Python: Sockets and HTTP Requests

Introduction to Computer Programming

Management and Artificial Intelligence

LUISS

Network Communication

So far...

- we know how to read and write files on our local disk;
- we know how to process data stored in our computer;
- we know how to make our programs persistent.

However, in the modern world, most applications need to communicate over the network:

- web browsers fetch pages from servers
- mobile apps retrieve data from APIs
- IoT devices send sensor data to the cloud

Today we'll learn how Python programs can communicate over the Internet!

Today let's answer the following questions:

1. What are sockets and how do they enable network communication?
2. How can we make HTTP requests to fetch data from the web?
3. What is JSON and why is it so popular for data exchange?
4. How can we use REST APIs to interact with web services?

Focus on:

1. Basic socket programming concepts
2. The `requests` library for HTTP
3. JSON serialization and deserialization
4. Practical examples with real-world APIs

Sockets

A **socket** is an endpoint for sending or receiving data across a network.

Think of it as a "phone connection" between two computers:

- One computer (server) listens for incoming connections
- Another computer (client) initiates the connection
- Once connected, they can exchange data

Python provides the `socket` module for low-level network programming.

Sockets

A **socket** is an endpoint for sending or receiving data across a network.

Think of it as a "phone connection" between two computers:

- One computer (server) listens for incoming connections
- Another computer (client) initiates the connection
- Once connected, they can exchange data

Python provides the `socket` module for low-level network programming.

> **NOTE**: Sockets are the foundation of all network communication. HTTP, email, file transfer - they all use sockets under the hood!

Simple Socket Example

```
import socket

# Create a socket object
mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Connect to a web server on port 80 (HTTP)
mysock.connect(('data.pr4e.org', 80))

# Send an HTTP request
cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\r\n\r\n'.encode()
mysock.send(cmd)
```

```
47
```

```
# Receive data
while True:
    data = mysock.recv(512)
    if len(data) < 1:
        break
    print(data.decode(), end='')

mysock.close()
```

```
HTTP/1.1 200 OK
Date: Mon, 03 Nov 2025 18:47:15 GMT
Server: Apache/2.4.52 (Ubuntu)
Last-Modified: Sat, 13 May 2017 11:22:22 GMT
ETag: "a7-54f6609245537"
Accept-Ranges: bytes
Content-Length: 167
Cache-Control: max-age=0, no-cache, no-store, must-revalidate
Pragma: no-cache
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Connection: close
Content-Type: text/plain

But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
```

NOTE: In the code above:

- `AF_INET` means we're using IPv4 addressing
- `SOCK_STREAM` means we're using TCP (reliable, ordered communication)
- Port 80 is the default port for HTTP
- The `.encode()` method converts strings to bytes (sockets work with bytes)
- The `.decode()` method converts bytes to strings

HTTP Requests

Making HTTP requests with raw sockets is complex. We have to manually format the HTTP request, handle headers, decode responses, etc.

Luckily, Python has libraries that make this much easier!

The most popular is the `requests` library, which provides a simple, intuitive interface for making HTTP requests.

Installing `requests`

The `requests` library is not part of Python's standard library. You need to install it:

```
pip install requests
```

Or if you're using conda:

```
conda install requests
```

Making GET Requests

The most common HTTP operation is GET - fetching data from a server.

Making GET Requests

The most common HTTP operation is GET - fetching data from a server.

```python
import requests

# Make a GET request
response = requests.get('http://data.pr4e.org/romeo.txt')

# The response object contains:
print(response.status_code)  # HTTP status code (200 = success)
print(response.text[:50])    # Response body as text
print(response.headers)      # Response headers
```

```
200
But soft what light through yonder window breaks
I
{'Date': 'Mon, 03 Nov 2025 18:51:22 GMT', 'Server': 'Apache/2.4.52 (Ubunt
u)', 'Last-Modified': 'Sat, 13 May 2017 11:22:22 GMT', 'ETag': '"a7-54f6609
245537"', 'Accept-Ranges': 'bytes', 'Content-Length': '167', 'Cache-Contro
l': 'max-age=0, no-cache, no-store, must-revalidate', 'Pragma': 'no-cache',
'Expires': 'Wed, 11 Jan 1984 05:00:00 GMT', 'Keep-Alive': 'timeout=5, max=1
00', 'Connection': 'Keep-Alive', 'Content-Type': 'text/plain'}
```

HINT: Common HTTP status codes:

- **200**: OK - request succeeded
- **404**: Not Found - resource doesn't exist
- **500**: Server Error - something went wrong on the server
- **401**: Unauthorized - authentication required

JSON: The Universal Data Format

JSON (JavaScript Object Notation) is a lightweight data-interchange format that has become the standard for exchanging data on the web.

JSON: The Universal Data Format

JSON (JavaScript Object Notation) is a lightweight data-interchange format that has become the standard for exchanging data on the web.

Why is JSON so popular?

- It's human-readable
- It's language-independent
- It can represent complex data structures
- It's easy for machines to parse and generate

JSON Syntax

JSON data types map to Python data types:

JSON	Python
`string`	`str`
`number`	`int` or `float`
`boolean`	`bool`
`null`	`None`
`array`	`list`
`object`	`dict`

Example of JSON data:

```
{
    "name": "Alice",
    "age": 30,
    "is_student": false,
    "courses": ["Math", "Physics", "CS"],
    "address": {
        "street": "123 Main St",
        "city": "Rome"
    }
}
```

Working with JSON in Python

Python's `json` module (built-in!) provides functions to convert between JSON strings and Python objects:

Working with JSON in Python

Python's `json` module (built-in!) provides functions to convert between JSON strings and Python objects:

```python
import json

# Python dictionary
data = {
    "name": "Alice",
    "age": 30,
    "courses": ["Math", "Physics"]
}

# Convert Python dict to JSON string
json_string = json.dumps(data)
print(json_string)

# Convert JSON string back to Python dict
data_again = json.loads(json_string)
print(data_again)
```

```
{"name": "Alice", "age": 30, "courses": ["Math", "Physics"]}
{'name': 'Alice', 'age': 30, 'courses': ['Math', 'Physics']}
```

NOTE: Remember the key functions:

- `json.dumps()`: Python object → JSON string (dumps = "dump string")
- `json.loads()`: JSON string → Python object (loads = "load string")
- `json.dump()`: Python object → JSON file
- `json.load()`: JSON file → Python object

Reading and Writing JSON Files

```python
import json

# Write JSON to file
data = {"name": "Bob", "age": 25}
with open("person.json", "w") as f:
    json.dump(data, f)

# Read JSON from file
with open("person.json", "r") as f:
    loaded_data = json.load(f)
    print(loaded_data)  # {'name': 'Bob', 'age': 25}
```

REST APIs: Interacting with Web Services

REST (Representational State Transfer) is an architectural style for designing web services.

Most modern web APIs follow REST principles:

- Resources are identified by URLs
- HTTP methods (GET, POST, PUT, DELETE) specify the operation
- Data is exchanged in JSON format
- Each request is independent (stateless)

Fetching Data from a Public API

Let's start with a simple public API that provides interesting data:

```
import requests

# Fetch a random joke from a public API
response = requests.get('https://official-joke-api.appspot.com/random_joke')

# Check if successful
if response.status_code == 200:
    joke = response.json()  # Automatically parses JSON!
    print(f"Setup: {joke['setup']}")
    print(f"Punchline: {joke['punchline']}")
else:
    print(f"Error: {response.status_code}")
```

```
Setup: What do you call a bee that can't make up its mind?
Punchline: A maybe.
```

HINT: The `requests` library makes it incredibly easy to work with JSON APIs! The `.json()` method automatically converts the JSON response into a Python dictionary.

Working with APIs that Require Authentication

```
```

```
Error: 429
```